ML FRIDAYS

Fine tuning LLMs

Sudhanshu Hate

Principal Architect - AI ML Specialist
Amazon Web Services

Agenda

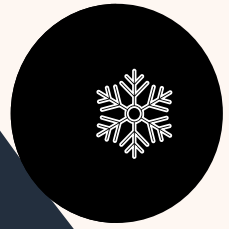
- Instruction fine-tuning
- Fine-tuning on a single Task
- Multi-task instruction fine tuning
- Parameter efficient fine tuning

  LoRA, QLoRA
- PEFT technique - Soft prompts
- Example walk-through
- Fine-tuning with reinforcement learning

In-context learning

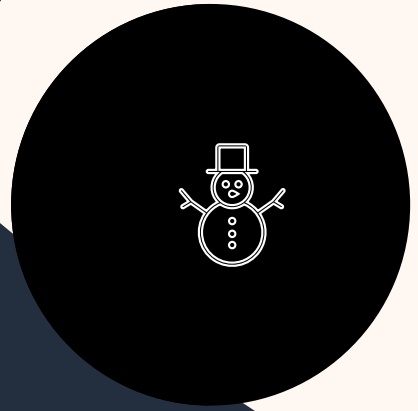
- Example prompt -> Model -> Completion

LLM Customizations

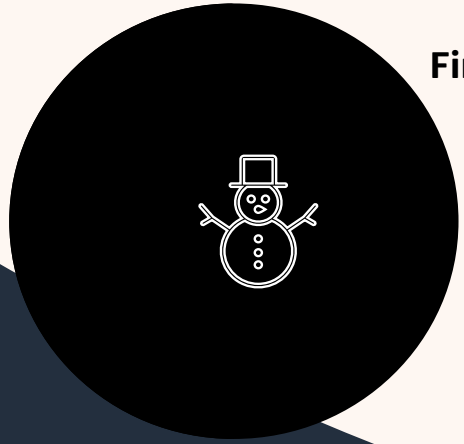


Right LLM with prompting techniques

In-context learning and RAG



LLM Agents



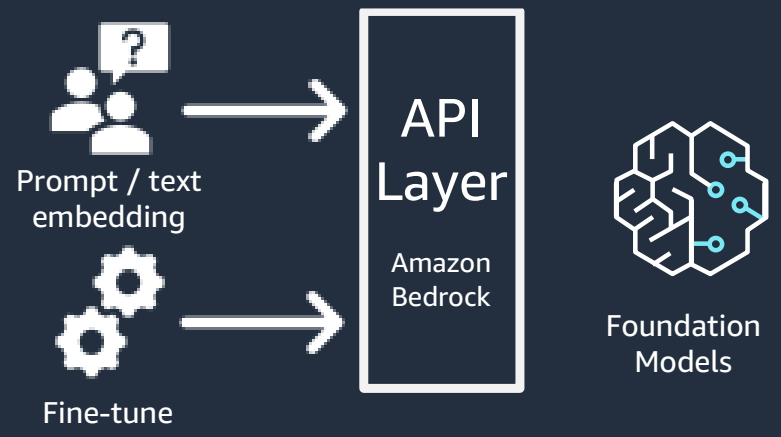
Fine-tuning/RLHF

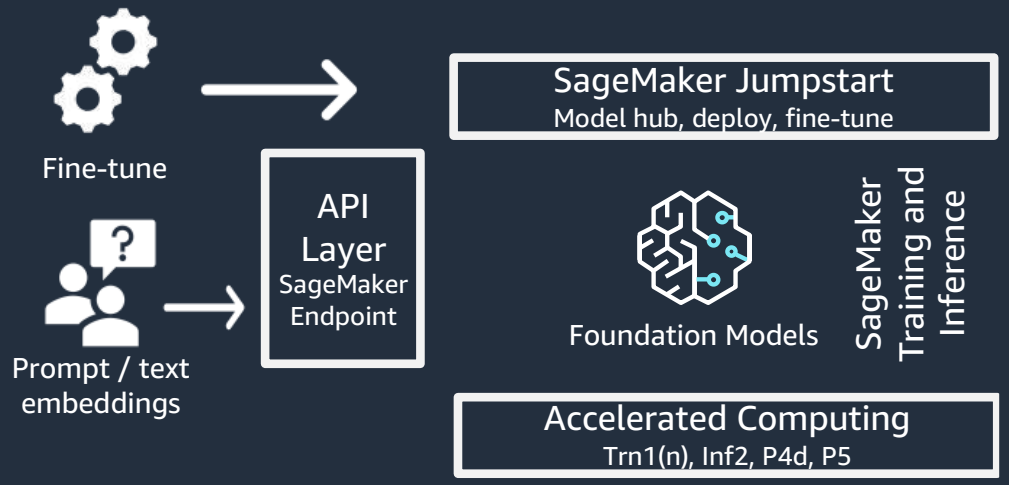
Pre-training or Build your own Model

Choosing right foundation models

- Commercial & license
- Speed and precision
  - Number of parameters
- Context length and model size
- Task-specific vs general-purpose
- Transformer architectures
- Deployment cost

How do I access foundation models?





Amazon Bedrock

- The easiest way to build and scale generative AI applications with FMs
- Access directly or fine-tune foundation model using API
- Serverless

Amazon SageMaker JumpStart

- ML hub with FMs, built-in algorithms, and prebuilt ML solutions that you can deploy with just a few clicks
- Deploy FM as Amazon SageMaker endpoint (hosting)
- Fine-tuning leverages Amazon SageMaker training jobs
- Choose Amazon SageMaker managed accelerated computing instance

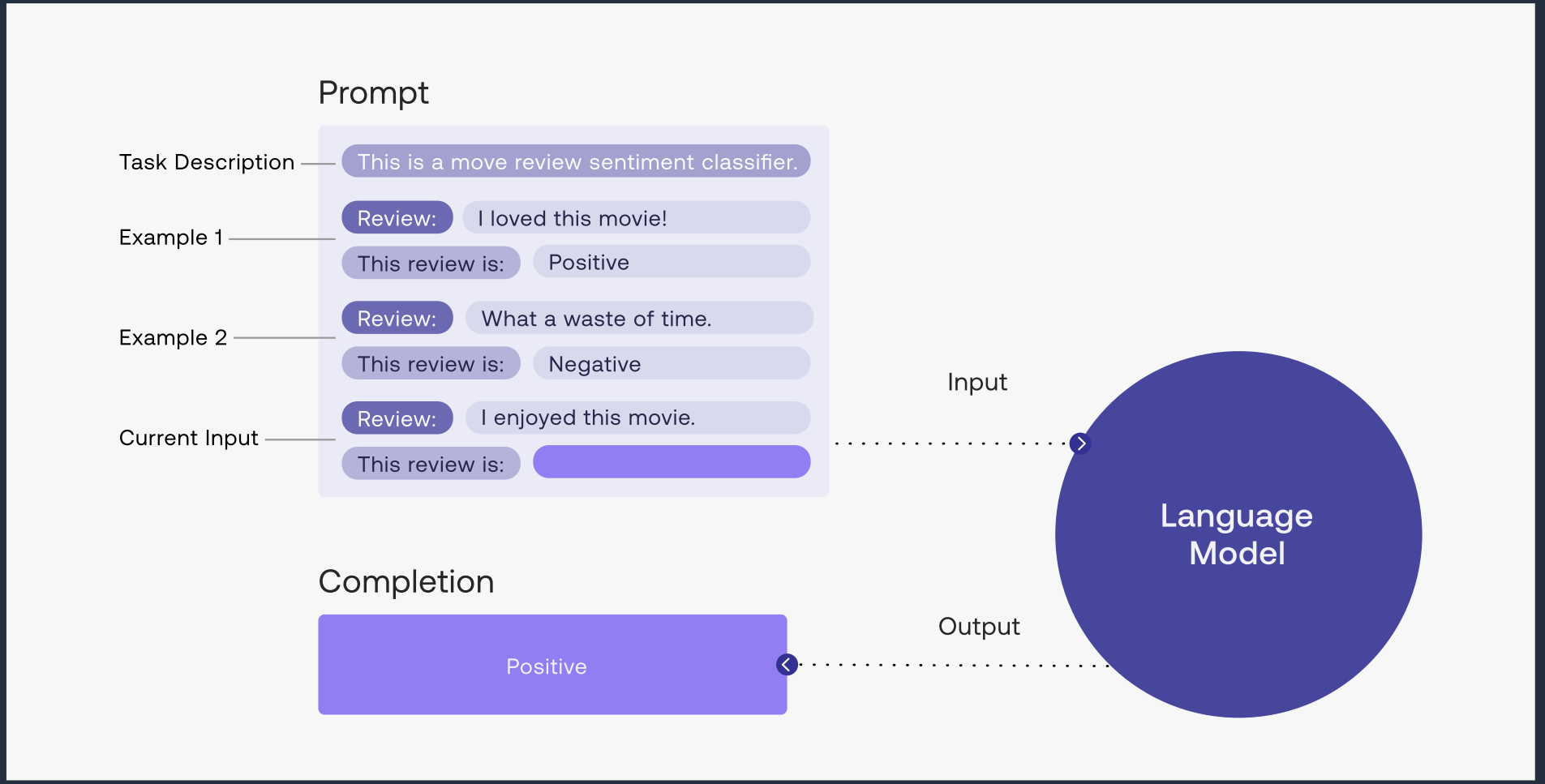
Prompt engineering techniques

Prompt engineering

In-context learning – Zero shot Inference

Prompt engineering

In-context learning – few shot learning

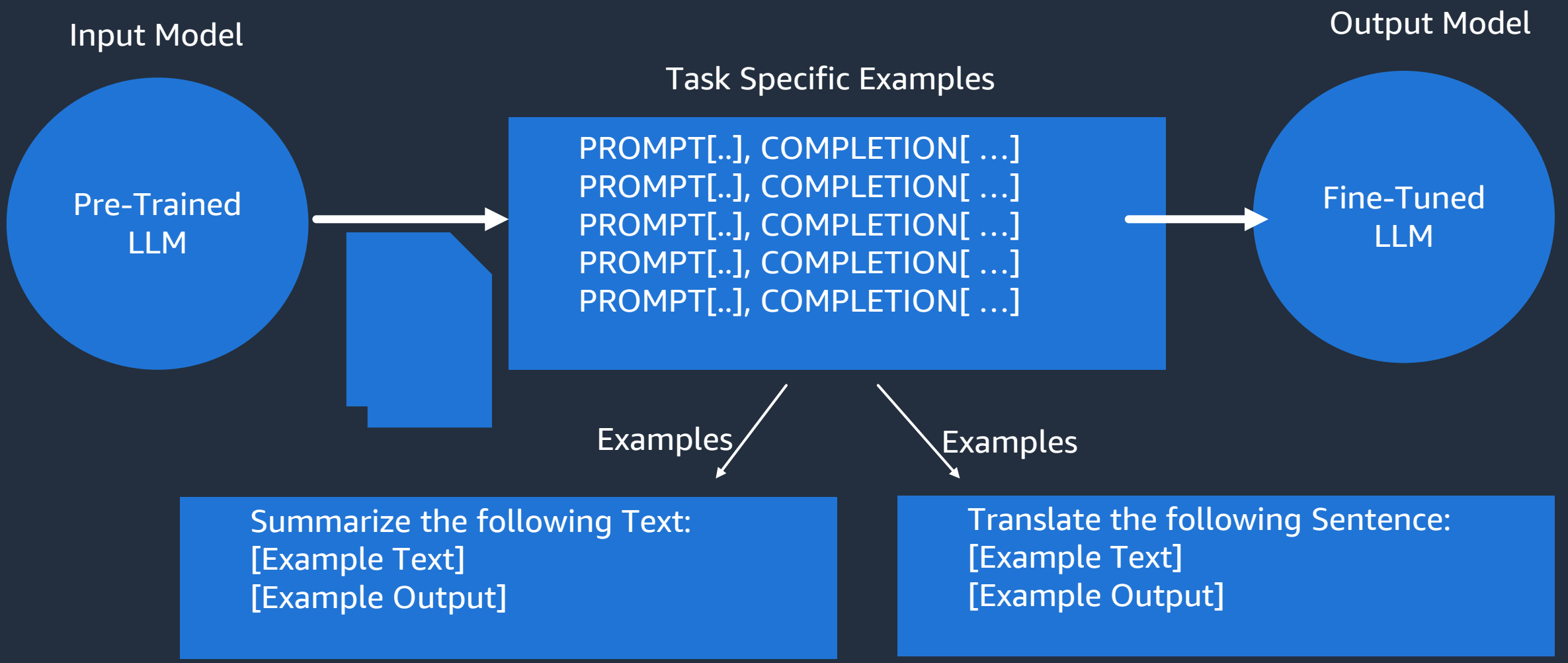


* Image courtesy: Cohere.ai

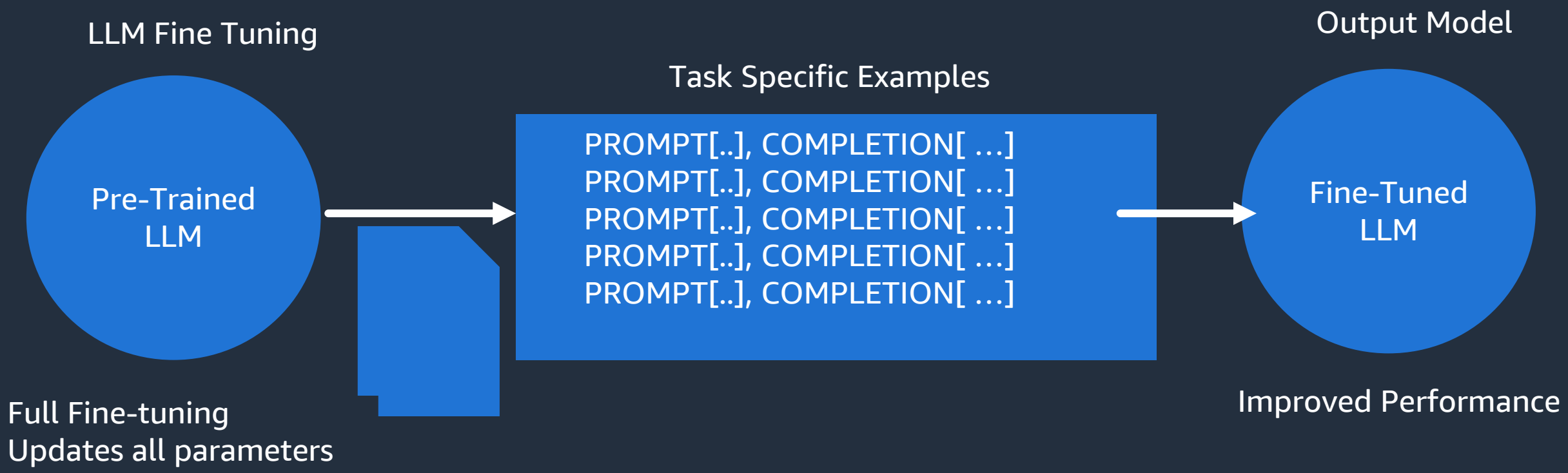
Limitations of in-context learning

- Constrained by context window size
- May not work for smaller models
- At times, outputs are not reliable, correct

LLM fine tuning at a high level

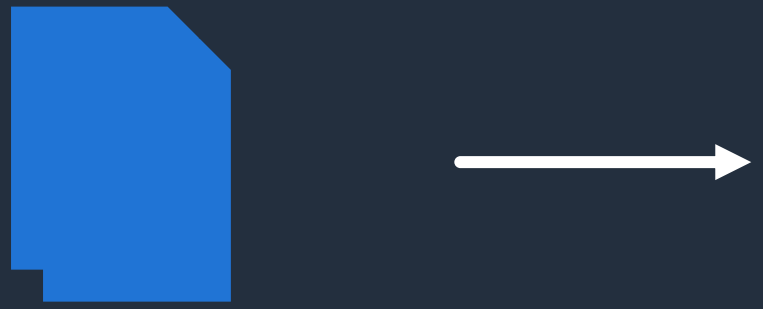


Using prompts to fine tune LLM with instructions



LLM fine tuning process

LLM Fine Tuning



Instruction dataset (prepared)

Training data splits

PROMPT[..], COMPLETION[...]
PROMPT[..], COMPLETION[...]
PROMPT[..], COMPLETION[...]
PROMPT[..], COMPLETION[...]
PROMPT[..], COMPLETION[...]

Training data

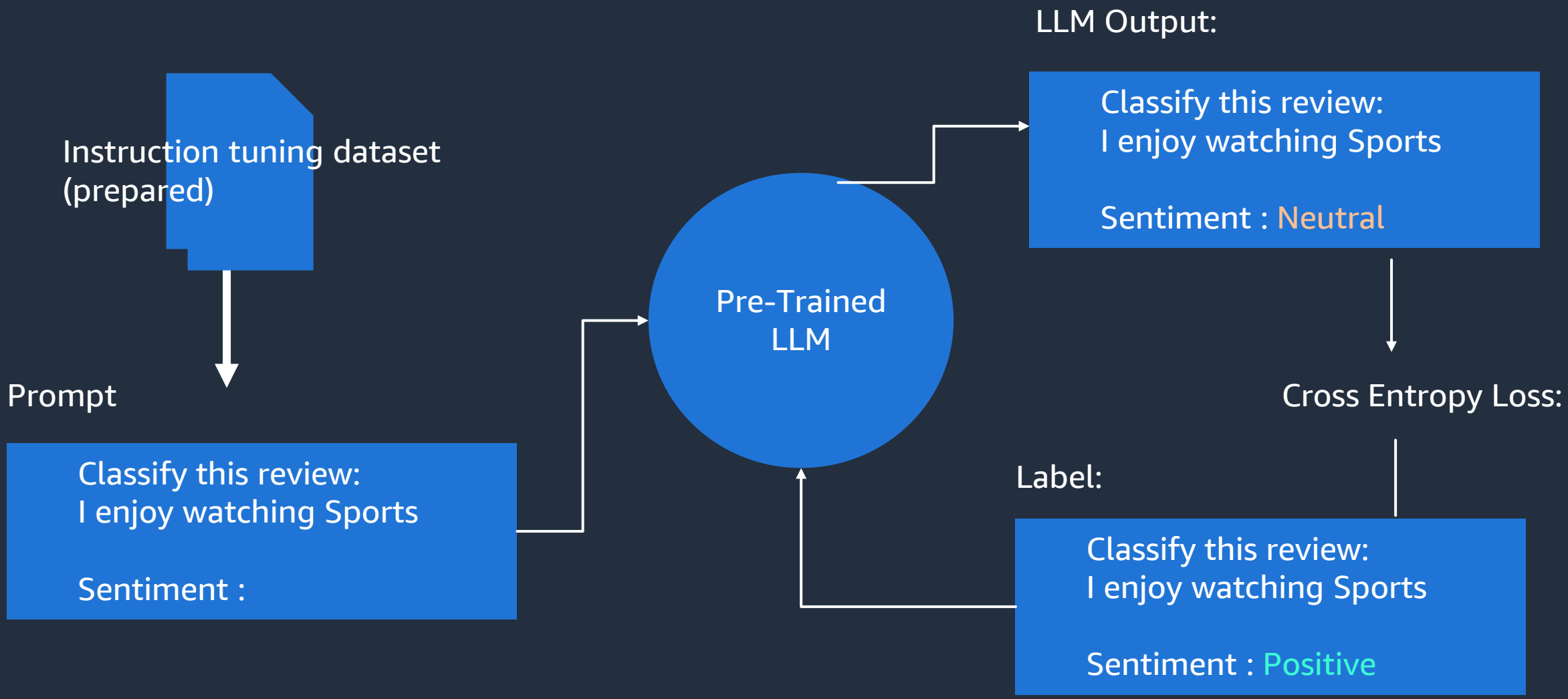
PROMPT[..], COMPLETION[...]
PROMPT[..], COMPLETION[...]
PROMPT[..], COMPLETION[...]

Validation data

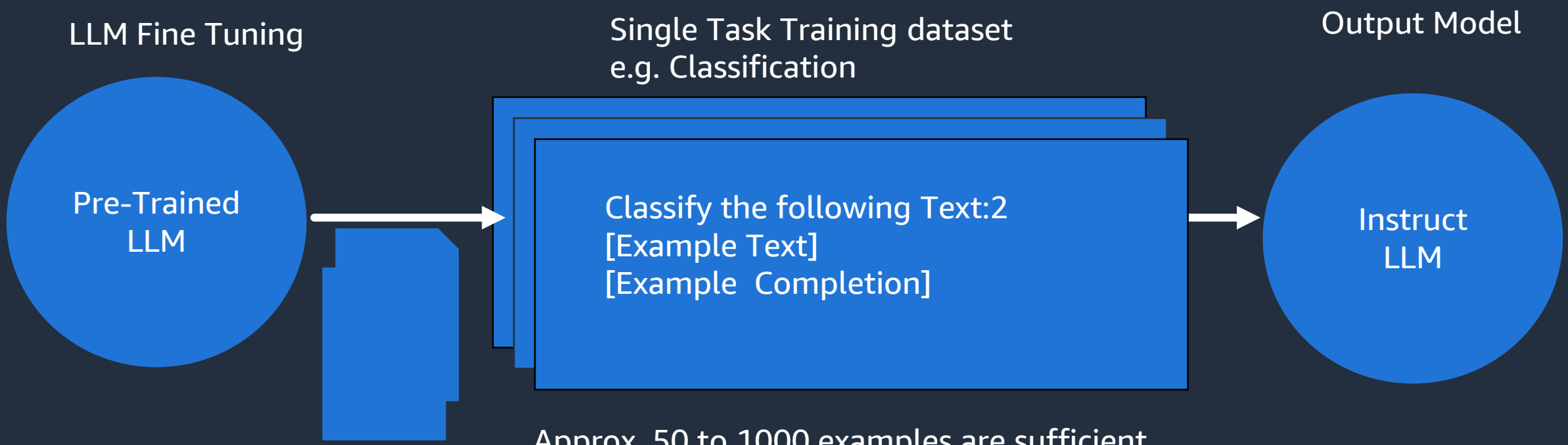
PROMPT[..], COMPLETION[...]
PROMPT[..], COMPLETION[...]

Test data

LLM Fine-tuning process



Fine tuning on a single task



Catastrophic forgetting

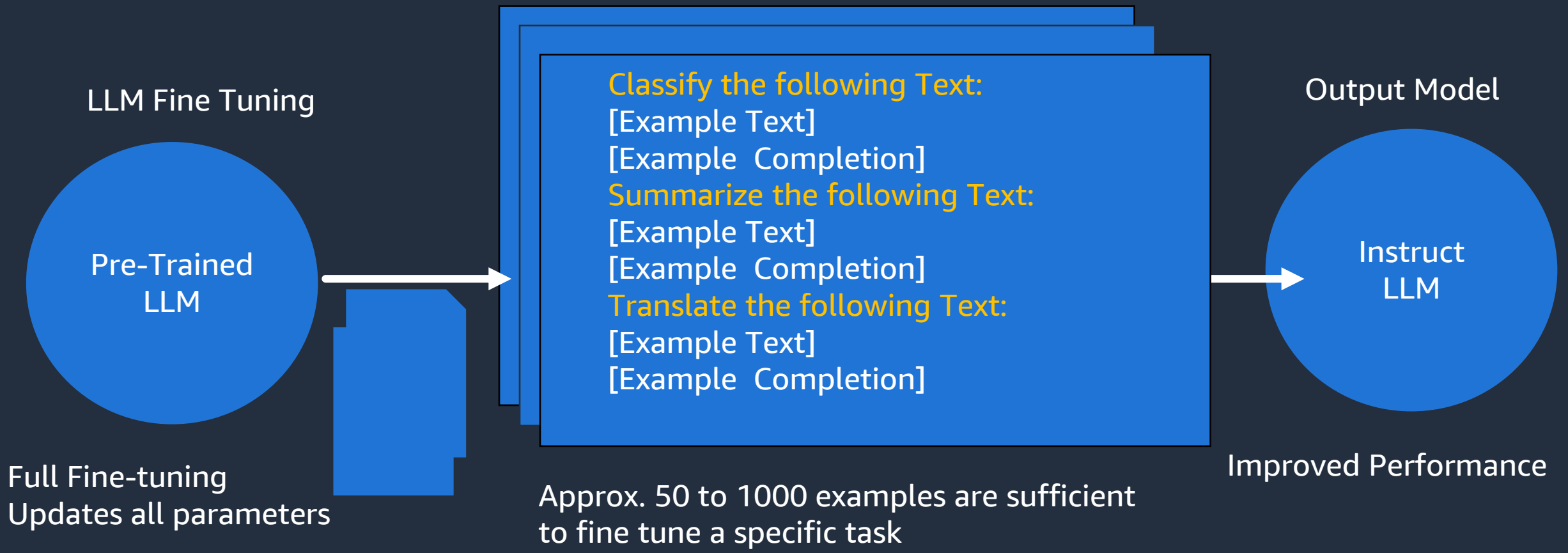
- For a multi-task LLM, fine tuning on a specific task, can significantly increase the performance of a model on a single task

- Side effect - Other existing tasks performance can significantly deteriorate

How to avoid catastrophic forgetting?

- Fine - tune on multiple tasks at the same time
- Consider Parameter Efficient Fine-tuning (PEFT)

Fine tuning on a multiple task(s)



LLM evaluation - Challenges

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

LLM evaluation - Challenges

- Matching no. of words
- Word meaning differs
- Evaluating summaries

(qualitative)

e.g. 1

- I enjoy trekking
- I adore going on to a trek

e.g. 2

- I do not enjoy much of an action movie
- I prefer to watch romantic movies

LLM evaluation - Metrics

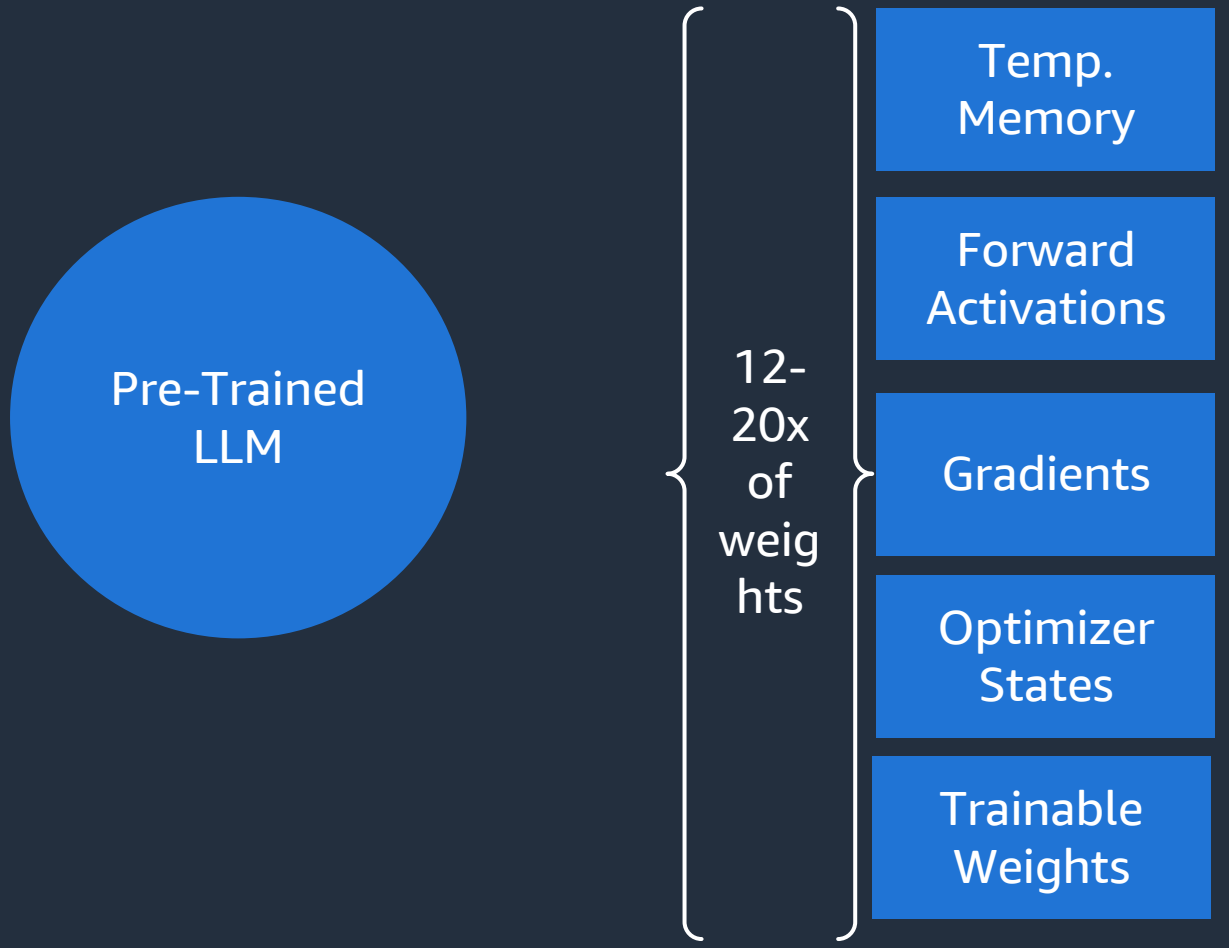
Rogue

- used for text summarization
- Compares a summary to one or more reference summaries

Blue Score

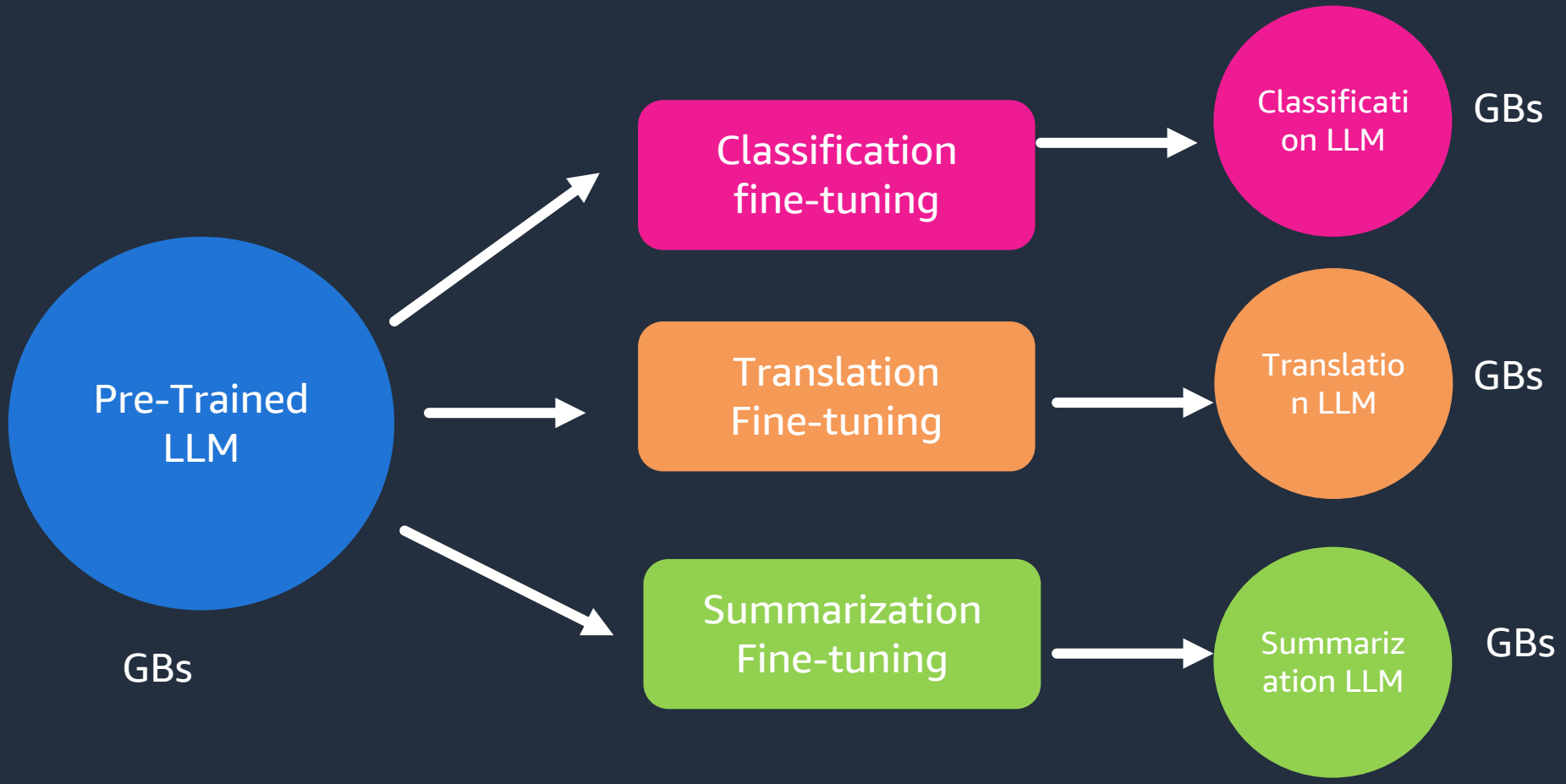
- Used for text translation
- Compares to human generated text translations

Full fine tuning of LLM is challenging



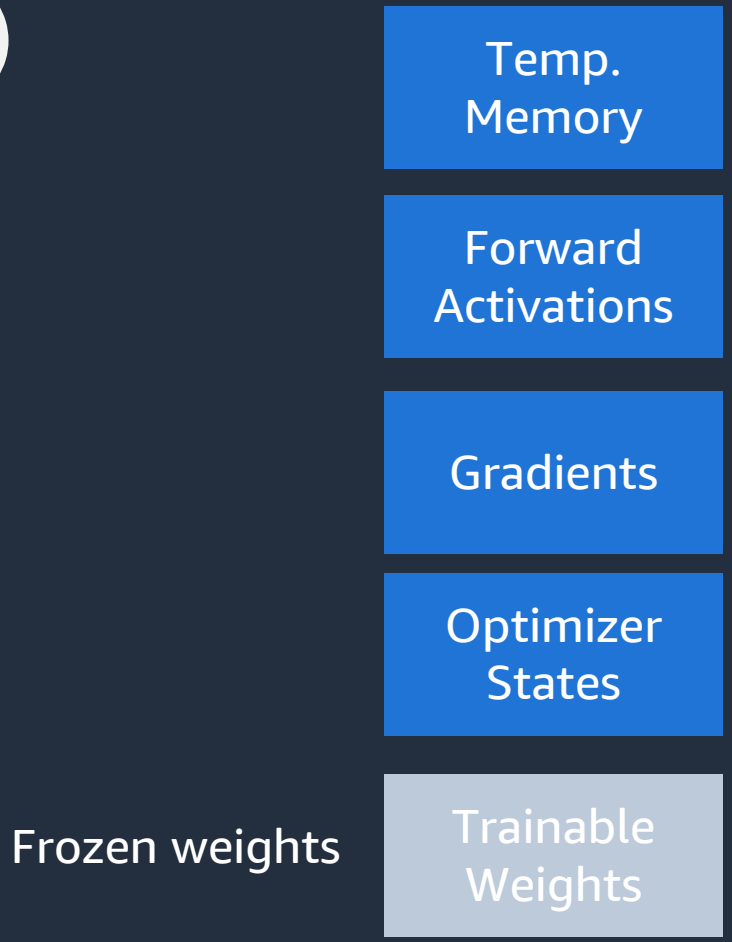
Full fine tuning

- Full Fine-tuning creates full copy of original LLM task



Parameter efficient fine-tuning (PEFT)

LLM with additional layers for PEFT



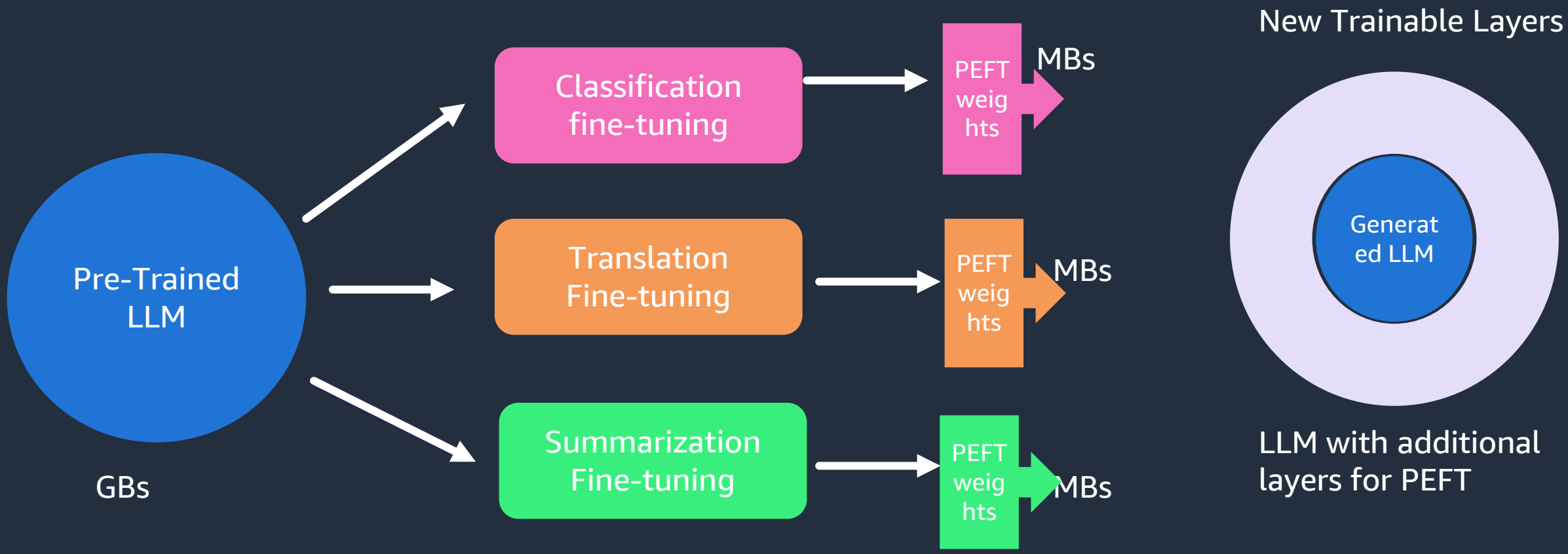
Frozen weights

Advantages

- Trains only small subset of parameters
- Less Prone to Catastrophic Forgetting
- Only occupies about 20% of memory

PEFT – saves space and is flexible

- Full Fine-tuning creates full copy of original LLM task



PEFT methods

Selective

- Selects subset of initial LLM parameters to fine-tune

Reparameterization

- Reparametrize model weights using low-rank representation

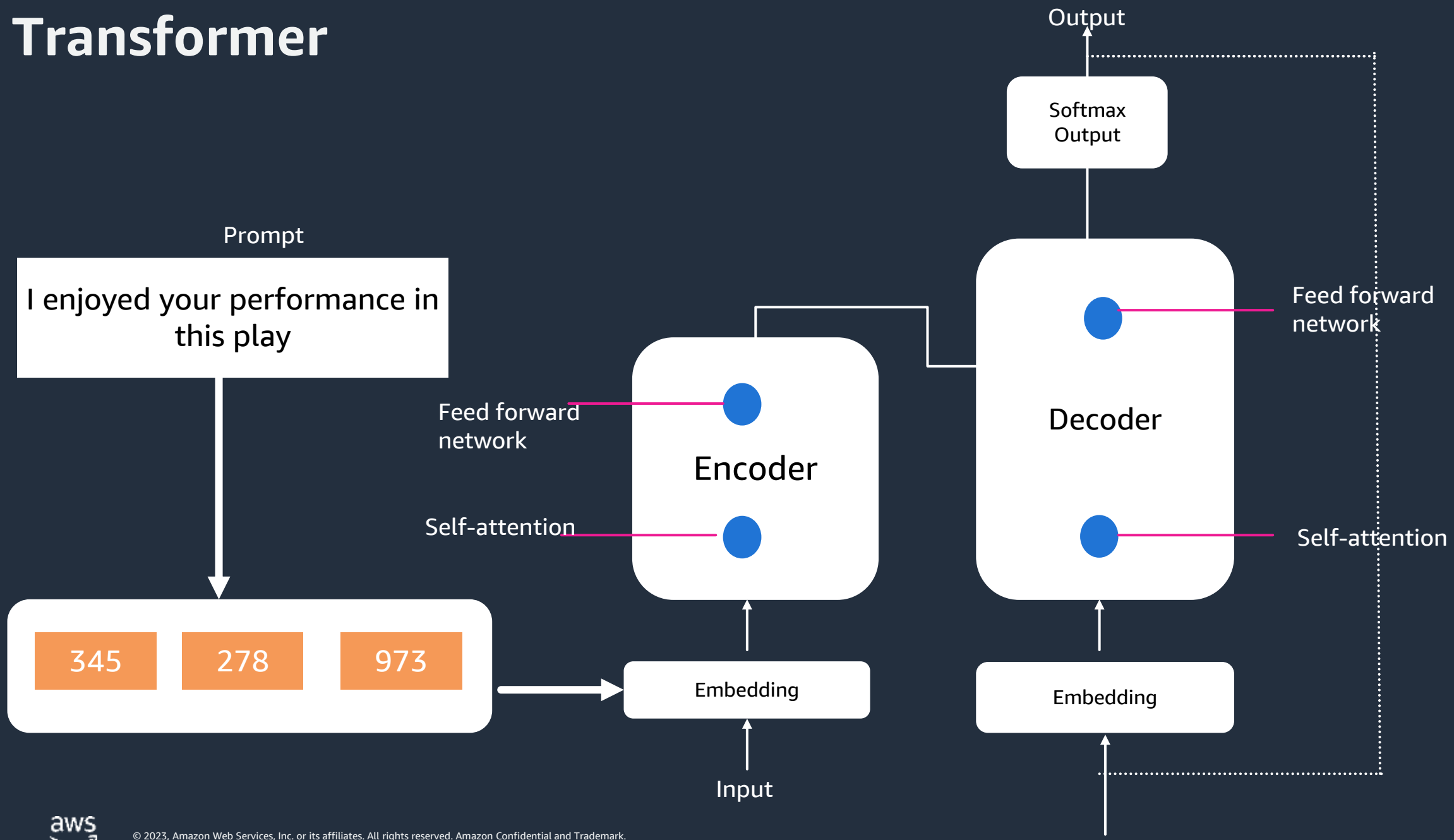
Additive

- Add trainable layers or parameters to model
  - Adapters
  - Soft Prompts – Prompt tuning

PEFT tradeoffs

- Parameter efficiency
- Memory efficiency
- Training Speed
- Model performance
- Inference costs

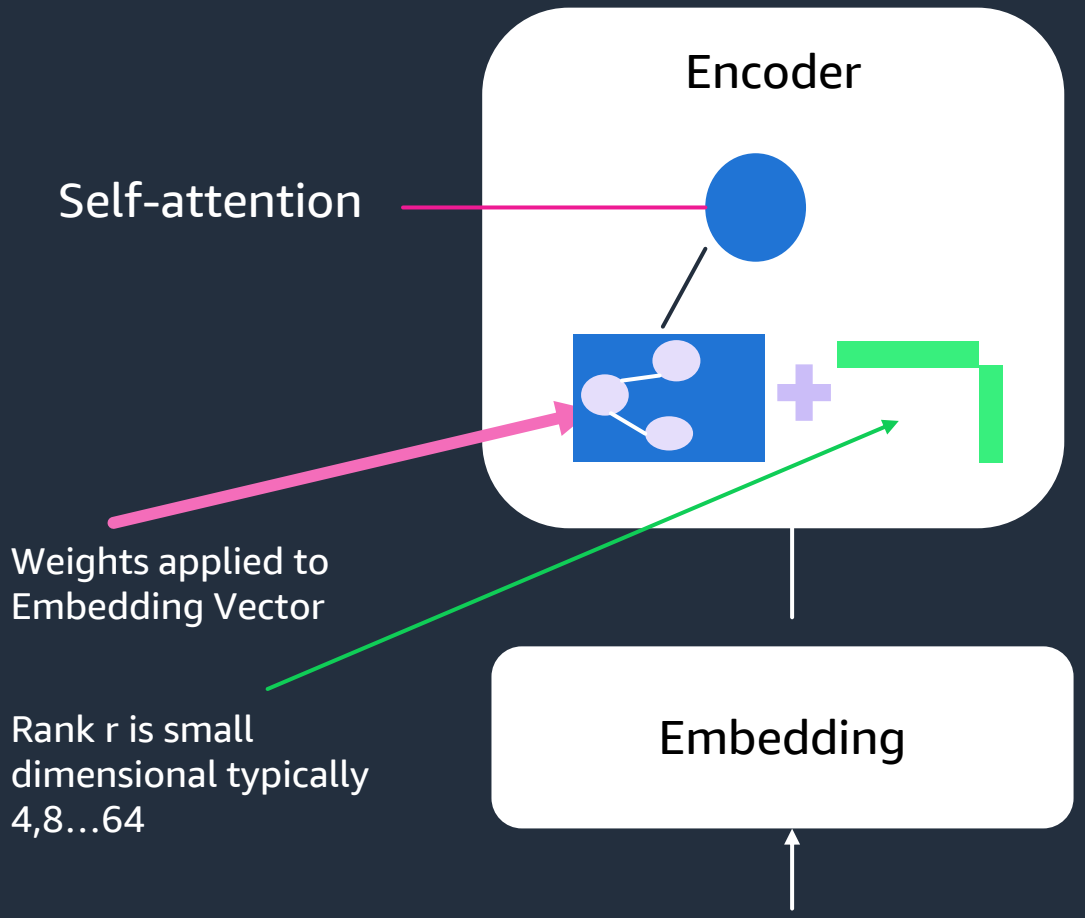
Transformer



LoRA – Low rank adaptation of LLMs -steps

- Input prompt is converted into tokens which in turn is converted into embedding vector
- Embeddings are fed to encoder and decoder network of a transformer
- Each of the encoder and decoder layer has two types of Neural networks - self attention, and feed forward neural network. The weights of these networks were learnt during the pre-training process
- The weights are fed to the self attention layer where series of weights are applied to embedding vector to calculate the attention score
- LoRA freezes most of the original model weights and injects the 2 Rank decomposition matrices alongside original weights
- The dimensions of small matrix (Rank matrix) is kept in such a manner that the product of resultant matrix is of the same dimension as the original matrix
- Rank matrix is typically of small dimension of 2, 4, 8…64
- Now, train the smaller matrices using the same supervised learning approach
- For Inference, 2 low Rank matrix are multiplied together to create the matrix of the same dimension as the frozen weights. Add this to original weight and replace them with the model with the updated value
- You have a LoRA fine tuned model that can carry out specific task

LoRA – Low rank adaptation of LLMs



1. Freezes most of the original LLM weights
2. Inject 2 Rank decomposition matrices
3. Training the weights of smaller matrices

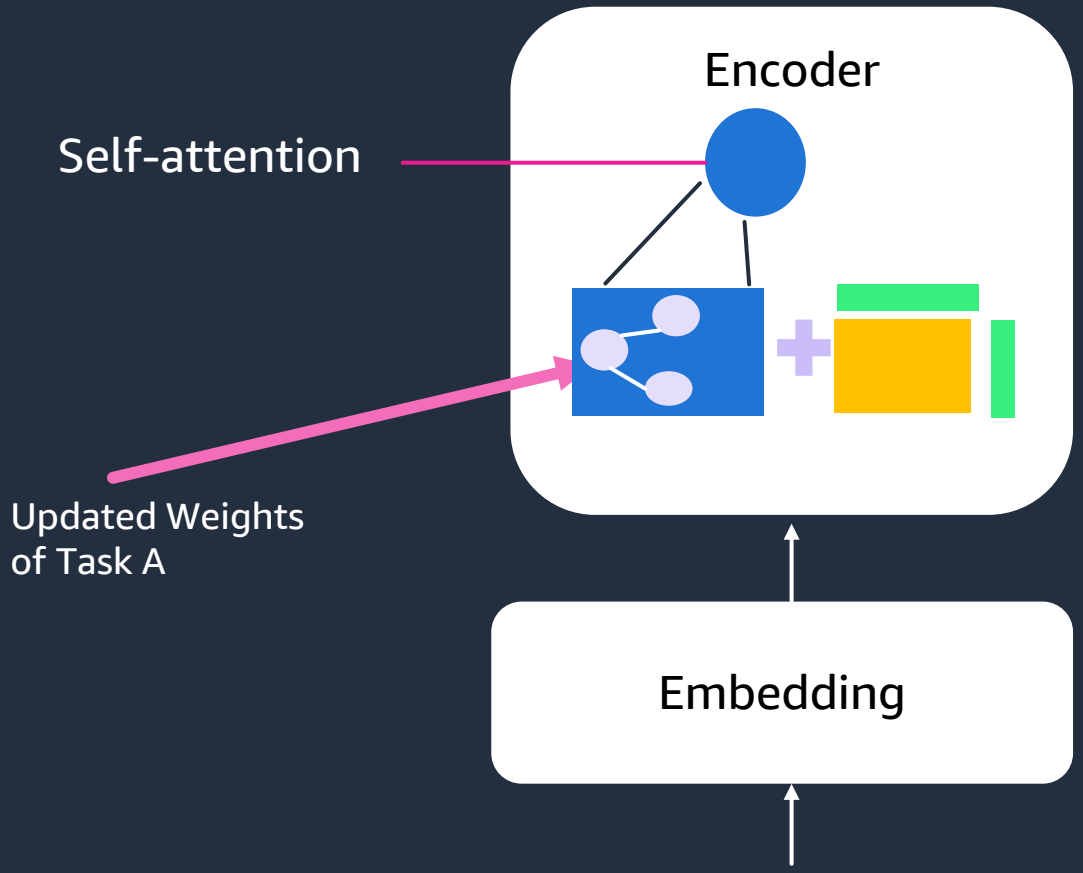
<u>Steps to update model for Inference</u>

1. Matrix multiply the low rank matrices

2. Add to Original weights

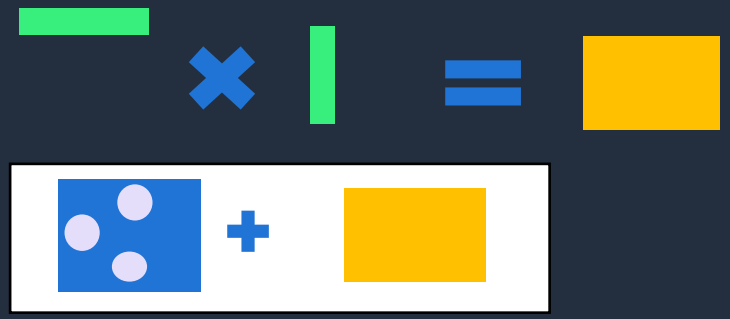
Updated weights (New)

LoRA – Low rank adaptation of LLMs

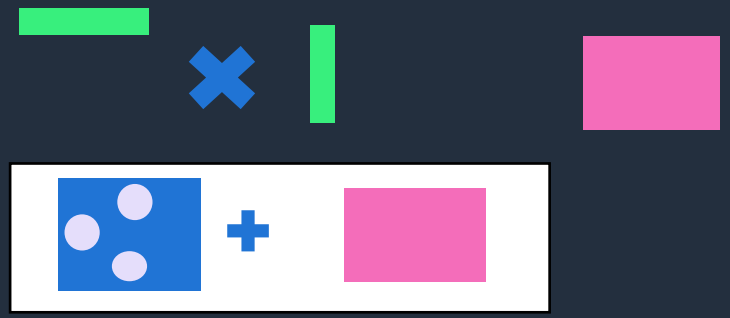


1. Train different rank composition matrices for different tasks
2. Update weights before Inference

Task A



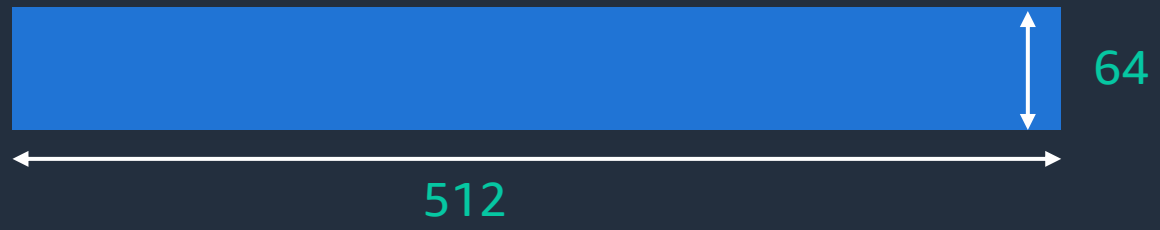
Task B



Example using transformer as reference

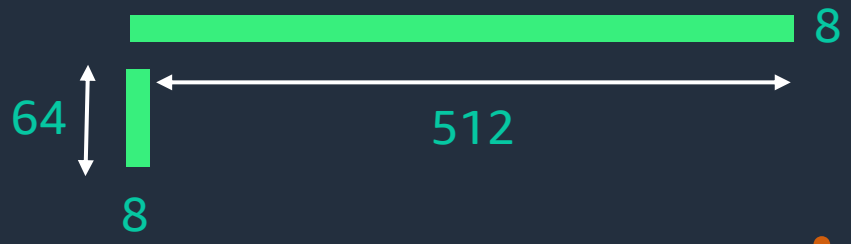
Use the base Transformer model presented by Vaswani et.all 2017

- Transformer weights have dimensions d*k = 512*64 = 32,768 trainable parameters



In LoRA with rank r= 8:

- A has dimension r*K = 8*64 = 512 parameters
- B has dimension d*r = 512*8 = 4096 trainable parameters
- Total parameters = 512 + 4096 = 4608
- Parameter reduction = (4608/32768) *100 = 85.93%



- 86% reduction in parameters to train

QLoRA – Quantized LoRA

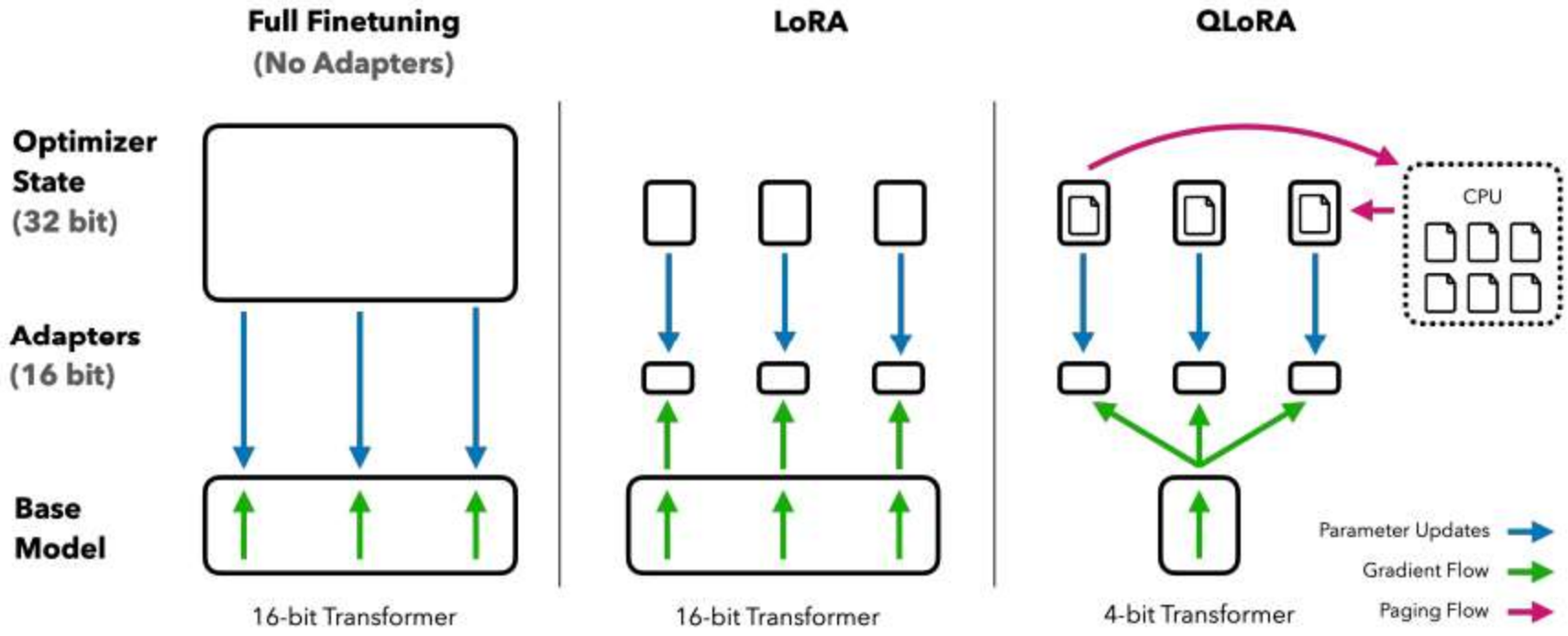


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

QLoRA – Quantized LoRA

- Introduces 4 bit normal float (nf4) data type for 4-bit quantization
- Supports double quantization to reduce memory ~ 0.4 bits per parameter (~3Gb for a 65B parameter model)
- Unified GPU-CPU memory management reduces GPU memory usage
- LoRA adapters at every layer - not just attention layers
- Minimizes accuracy trade offs

PEFT Technique - Soft Prompts

- **Fine-tuning with Reinforcement learning**

Demo – Notebook walkthrough

Summary

References

QLoRA https://arxiv.org/abs/2305.14314

-

Thank you!

Sudhanshu Hate
HiSuds@amazon.com